RAG Document Q&A - Demo Guide

Sample document bundled with the app.

1. What this demo is

This is a document question-answering assistant built with retrieval-augmented generation (RAG). It answers questions using only the PDFs it has been given. This guide is the sample document that is loaded automatically, so visitors can try the app without uploading anything. Upload your own PDFs in the sidebar to ask questions about them instead. Uploaded files are processed in memory and are not stored: they disappear when the session ends or the Space restarts.

2. Why retrieval-augmented generation exists

A large language model only knows what was in its training data. It has never seen your private reports, contracts or lecture notes, and its knowledge stops at a training cut-off date. Fine-tuning a model on your documents is expensive and goes stale as soon as a document changes. RAG avoids both problems: instead of changing the model, it looks up the relevant passages at question time and hands them to the model as context.

3. Indexing: done once per document

When a PDF is processed, each page is extracted as text with PyPDFLoader, keeping the page number so answers can cite it. The text is then split into chunks of about 1000 characters with an overlap of 200 characters between neighbouring chunks. The overlap exists so that a sentence sitting on a chunk boundary still appears complete in at least one chunk. Each chunk is converted into a 384-number vector by the all-MiniLM-L6-v2 embedding model, and the vectors are stored in a Chroma vector database.

4. Querying: done for every question

When you ask a question, the question is embedded with the same model, and Chroma returns the chunks whose vectors are closest by cosine similarity. With the Gemini generator the app retrieves the top 8 chunks; with the local generator it retrieves the top 4, because the local model can only read about 512 tokens. The chunks are placed into a prompt that tells the model to answer only from them, to cite page numbers, and to reply "I don't have enough information in these documents to answer that" when the chunks do not contain the answer.

5. The two generators

The default generator is Google Gemini (gemini-3.5-flash-lite) on the free tier. It gives the best answers, but the question and retrieved text are sent to Google, and the free tier has a daily request limit shared by everyone using this demo. The fallback generator is google/flan-t5-base, a 250-million-parameter model that runs on the CPU with no API key. Nothing leaves the server, but its answers are short and noticeably less reliable. If no Gemini key is configured, or the daily limit is reached, the app switches to the local model and says so.

6. Known limitations

Retrieval quality caps answer quality: the model cannot use a passage that was never retrieved. Scanned PDFs that contain images of text, rather than real text, will not work without OCR. The embedding model is

trained mainly on English and handles other languages poorly. Similarity scores cannot reliably tell whether a question is answerable: in testing, an unanswerable question scored higher than several answerable ones.

7. A fact mentioned only once

The project mascot is a capybara named Chunky. This sentence exists to test whether a single, unrelated fact can still be found by retrieval.